

第九章附录：实战代码详细讲解

这份附录是第 9 章实战 notebook (09.ipynb) 的逐行导读，面向不太熟悉 Python / PyTorch 的读者：按代码的功能块讲清楚每个 cell 做了什么，入参、出参、中间变量分别是什么，形状怎么一步步变。正文的「为什么这样设计」在第九章：Transformer 整体架构里，这里专注「代码每一行在干什么」，两份对照着读最顺。

想直接跑示例？点这里  [Open in Colab](#)。

硬件门槛：概念章，CPU 即可 ✓。本章只在维度 ≤ 128 的小张量上跑，最大的开销是 Cell 5 那个 24 层小模型训练对照实验，Colab 免费 CPU 运行时约一两分钟跑完，不需要 GPU；Cell 6 读 Qwen3-8B config 只下载几 KB 的 json。

整本 notebook 的主线（Cell 0-1 是环境自检 + 装依赖，无须细讲，本附录略过）：把第 6-8 章造好的零件（RMSNorm、RoPE、因果 GQA、SwiGLU）用代码复现一遍 → 拼成一个 `DecoderBlock` → 外面包上 embedding / final norm / lm_head 拼成完整 `MiniDecoderLM` → 前向一遍验证「逐层形状守恒」并数参数 → 用 24 层深模型做 Pre-LN / Post-LN 的训练稳定性对照 → 最后读真实 Qwen3-8B 的 config，按它的尺寸把参数明细重算一遍、对上全景图。

Cell 2：复现四个零件，拼出 `DecoderBlock`

这是全章的「零件车间」。第 6-8 章我们分别讲过 Transformer 的几个核心部件，这个 cell 把它们用最短的代码各复现一遍——RMSNorm（第 8 章的归一化）、RoPE（第 4 章的旋转位置编码）、因果 GQA（第 6、7 章的分组查询注意力）、SwiGLU（第 8 章的前馈网络）——再用它们拼出一个完整的 `DecoderBlock`（一个 Transformer 层）。本 cell 只定义类和函数，不跑任何前向，真正跑起来要等 Cell 3 / Cell 4。

先看 cell 顶部的 import：

```
# =====  
# Cell 2: 复现第 6-8 章的零件，并拼出一个 DecoderBlock  
# =====  
import torch  
import torch.nn as nn  
import torch.nn.functional as F  
import math
```

`torch` 是 PyTorch 主库；`torch.nn`（惯例简写 `nn`）装着各种神经网络层（`Linear`、`Embedding`、`Module` 等）；`torch.nn.functional`（简写 `F`）装着无参数的函数式算子（`silu`、`softmax` 等）；`math` 只用来取一个标量平方根 `math.sqrt`。

下面分四个零件 + 一个组装件逐个看。

一、RMSNorm：只缩放、不减均值的归一化

```
class RMSNorm(nn.Module):
    """只按均方根缩放、不减均值的归一化（第 8 章）。x: [..., d] -> [..., d]。"""
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(d)) # 每个特征一个可学习增益 γ
        self.eps = eps
    def forward(self, x):
        rms = x.pow(2).mean(-1, keepdim=True).add(self.eps).rsqrt() # 1/RMS, 形状 [..., 1]
        return x * rms * self.weight # 广播回 [..., d]
```

• 归一化（normalization）：把一层的激活值重新缩放到一个稳定的尺度，避免越往深处数值不断放大（或不断收缩）。LayerNorm 先减均值再除标准差；RMSNorm（Root Mean Square Norm）把「减均值」这步省了，只除以均方根 $RMS(x) = \sqrt{\frac{1}{d} \sum_i x_i^2}$ ——又快又稳，现代 LLM（LLaMA、Qwen 等）基本都用它。

`__init__`（构造时建零件）：

入参	含义
<code>d</code>	要归一化的特征维度（这里就是 <code>d_model</code> ）
<code>eps</code>	一个极小常数（默认 <code>1e-6</code> ），防止除零，数值更稳

- `self.weight = nn.Parameter(torch.ones(d))`：一个长度 `d` 的可学习增益 γ （代码里叫 `weight`），初始化为全 1。`nn.Parameter` 把一个张量登记成「模型参数」，会随训练更新。每个特征一个增益，让模型可以自行调节各维度的幅度。
- `self.eps = eps`：存下来，`forward` 里要用。

forward（前向计算）：入参 `x` 形状 `[..., d]`（前面多少个维度都行，最后一维是特征），出参同形状 `[..., d]`。逐步看那行链式调用 `x.pow(2).mean(-1, keepdim=True).add(self.eps).rsqrt()`：

- `x.pow(2)`：逐元素平方，形状不变 `[..., d]`。
- `.mean(-1, keepdim=True)`：沿最后一维（特征维）求平均，得均方 $\frac{1}{d} \sum_i x_i^2$ 。`keepdim=True` 保留那一维为长度 1，得 `[..., 1]`（方便后面广播）。
- `.add(self.eps)`：加上 `eps`，防止下一步对 0 开根号。
- `.rsqrt()`：reciprocal square root，即 $1/\sqrt{\cdot}$ ，一步算出 $1/RMS$ 。结果 `rms` 形状 `[..., 1]`。
- `return x * rms * self.weight`：`x`（`[..., d]`）逐元素乘 `rms`（`[..., 1]`，广播成 `[..., d]`）完成归一化，再逐元素乘可学习增益 `weight`（`[d]`，也广播）。

广播（broadcasting）：两个形状不完全相同的张量做逐元素运算时，PyTorch 自动把长度为 1（或缺失）的维「拉伸」成对方的长度。这里 `[..., 1]` 和 `[d]` 都被拉成 `[..., d]` 再相乘。

二、build_rope_cache: 预先算好每个位置的 cos / sin

```
def build_rope_cache(seq_len, head_dim, base=10000.0):
    """预先算好每个位置 × 每个频率的 cos/sin (第 4 章 RoPE)。返回两个 [seq_len, head_dim]。"""
    inv_freq = 1.0 / (base ** (torch.arange(0, head_dim, 2).float() / head_dim)) # [head_dim/2]
    pos = torch.arange(seq_len).float() # [L]
    ang = torch.outer(pos, inv_freq) # [L, head_dim/2]
    cos = torch.cat([ang.cos(), ang.sin()], dim=-1) # [L, head_dim]
    sin = torch.cat([ang.sin(), ang.cos()], dim=-1) # [L, head_dim]
    return cos, sin
```

- RoPE (Rotary Position Embedding, 旋转位置编码, 第 4 章) : 给注意力注入「位置信息」的一种办法。它不往输入里加一个位置向量, 而是按 token 所在位置, 把 Q / K 向量在每一对维度上旋转一个角度——位置越靠后转得越多。两个 token 的相对距离就编码进了它们旋转角度之差里。要旋转就得先备好每个位置、每个频率对应的 cos 和 sin, 这个函数负责的就是预先把这张表算好。

入参: `seq_len` (要支持的最大序列长度 L)、`head_dim` (每个注意力头的维度 hd)、`base` (频率基数, 惯例 10000.0)。出参: 两个 `[seq_len, head_dim]` 的张量 `cos` / `sin`。

逐行看:

- `inv_freq = 1.0 / (base ** (torch.arange(0, head_dim, 2).float() / head_dim))`: 算一组频率。`torch.arange(0, head_dim, 2)` 是 `[0, 2, 4, ..., head_dim-2]` (步长 2, 共 `head_dim/2` 个), 除以 `head_dim` 再当 `base` 的指数、取倒数。用公式写, 第 i 个频率 $\theta_i = \text{base}^{-2i/d_{\text{head}}}$, 其中 i 从 0 数到 $d_{\text{head}}/2 - 1$ 、`base` 默认 10000、 d_{head} 是每头维度 hd 。得到的 `inv_freq` 形状 `[head_dim/2]`、第 i 个元素就是 θ_i : 维度越靠前 (i 越小) 指数越接近 0、 θ_i 越接近 1, 频率越高 (转得快); 越靠后 θ_i 越小, 频率越低 (转得慢)。这正是 RoPE 「不同维度用不同转速」的来历。
- `pos = torch.arange(seq_len).float()`: 位置下标 $p = 0, 1, \dots, L-1$, 形状 `[L]`。
- `ang = torch.outer(pos, inv_freq)`: 外积——`pos` (`[L]`) 和 `inv_freq` (`[head_dim/2]`) 两两相乘, 得 `[L, head_dim/2]`。即 `ang[p, i]` = 位置 p 在第 i 个频率上要旋转的角度 $p \cdot \theta_i = p \cdot \text{base}^{-2i/d_{\text{head}}}$ ——位置 p 越大、或频率 θ_i 越大, 要转的角度就越多。
- `cos = torch.cat([ang.cos(), ang.sin()], dim=-1)`: 对 `ang` 逐元素取余弦——即第 (p, i) 个元素是 $\cos(p \cdot \theta_i)$ ——得 `[L, head_dim/2]`, 再把它和自己沿最后一维拼一遍变成 `[L, head_dim]`。`sin` 同理, 元素是 $\sin(p \cdot \theta_i)$ 。

为什么 `cos` / `sin` 要把半截复制成两份拼成整截? 因为下面 `apply_rope` 的实现把 `head_dim` 维分成「前半 / 后半」配成旋转对, 前后两半共用同一组角度——所以这里先把 `[head_dim/2]` 的角度复制成 `[head_dim]`, 让 `cos` / `sin` 能和完整的向量逐元素相乘。

三、apply_rope: 把每个 head 的向量按位置旋转

```
def apply_rope(x, cos, sin):
    """把每个 head 的向量按位置旋转。x: [B, H, L, hd]; cos/sin: [L, hd]。"""
    cos, sin = cos[None, None], sin[None, None] # [1, 1, L, hd] 便于广播
```

```

d = x.shape[-1]
x1, x2 = x[..., : d // 2], x[..., d // 2:]          # 前一半 / 后一半配成旋转对
rotated = torch.cat([-x2, x1], dim=-1)              # 旋转 90° 的"虚部"
return x * cos + rotated * sin

```

入参： `x` (`[B, H, L, hd]` ——batch、头数、序列长、每头维度)； `cos` / `sin` (`[L, hd]`，上一函数打好的表)。出参：旋转后的 `x`，形状不变 `[B, H, L, hd]`。

数学上，把二维向量 (x_1, x_2) 旋转角度 θ 是：

$$\begin{pmatrix} x'_1 \\ x'_2 \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

即 $x'_1 = x_1 \cos \theta - x_2 \sin \theta$ 、 $x'_2 = x_2 \cos \theta + x_1 \sin \theta$ 。代码把这套「逐对旋转」用整段张量运算一次算完：

- `cos, sin = cos[None, None], sin[None, None]`： `[None, None]` 在最前面插两个长度 1 的维，`[L, hd] → [1, 1, L, hd]`，方便和 `x` (`[B, H, L, hd]`) 广播 (B、H 两维自动复制)。
- `d = x.shape[-1]`：取最后一维大小 `hd`。
- `x1, x2 = x[..., : d // 2], x[..., d // 2:]`：把向量沿最后一维切成前半 `x1` 和后半 `x2`，各 `[B, H, L, hd/2]`。这里把「第 `i` 维」和「第 `i + hd/2` 维」配成一个旋转对（而不是相邻两维配对——两种切法等价，这种「前后半」写法更省事）。
- `rotated = torch.cat([-x2, x1], dim=-1)`：构造旋转所需的「另一半」——把 `-x2` 和 `x1` 拼回 `[B, H, L, hd]`。对照旋转公式，这一项正是要乘 `sin` 的那部分。代码注释里说的「旋转 90°」就是这个意思：把每一对 $(x_j, x_{j+hd/2})$ 先逆时针转 90° 正好得到 $(-x_{j+hd/2}, x_j)$ ——对应到代码就是「把后半 `x2` 取负挪到前半位置、前半 `x1` 原样挪到后半位置」，正是 `cat([-x2, x1])` 干的事；再乘 `sin` 加到 `x cos` 上，合起来就是转 θ 角。
- `return x * cos + rotated * sin`：`x * cos` 对应 $x \cos \theta$ 、`rotated * sin` 对应那个带 `sin` 的交叉项，相加就完成了「每对维度旋转 θ 」。

`x[..., : d // 2]` 里的 `...` 是「省略号切片」，表示「前面所有维度原样保留，只对最后一维切片」。`//` 是整数除法 (`hd` 是偶数，正好分两半)。

注释里「虚部」是从复数视角打的比方：把一对 (a, b) 看成复数 $a + bi$ ，乘上 $e^{i\theta} = \cos \theta + i \sin \theta$ 就等价于旋转 θ 角；而乘虚数单位 i （相当于 $e^{i\pi/2}$ ）正好旋转 90°、把 (a, b) 变成 $(-b, a)$ ——代码里的 `rotated` 正是这一步乘 i 得到的「另一半」。不熟悉复数也没关系，按上面那个旋转矩阵理解就够了。

配一个最小数值例子：`head_dim=4`、3 个位置，把二、三连起来手算一遍

抽象的下标和广播看着费劲，不如把维度调到最小、手算一遍。取 `head_dim=4`（于是 `head_dim/2 = 2` 对旋转对）、`seq_len=3`（位置 0 / 1 / 2）、`base=100000`。

先跑 `build_rope_cache(seq_len=3, head_dim=4)`（第二部分那几行）：

- `torch.arange(0, head_dim, 2) = [0, 2]`，`/ head_dim = [0, 0.5]`，`base ** ... = [1, 100]`，取倒数得 `inv_freq = [1.0, 0.01]`（形状 `[2]`，正是两对的频率 $\theta_0 = 1.0$ 、 $\theta_1 = 0.01$ ）。
- `pos = [0, 1, 2]`（形状 `[3]`）。
- `ang = torch.outer(pos, inv_freq)`：每个位置 $\times$ 每个频率，得 `[3, 2]`，即 $\text{ang}[p, i] = p \cdot \theta_i$ ：

位置 p	$\theta_0 = 1.0$	$\theta_1 = 0.01$
0	0.0	0.00
1	1.0	0.01
2	2.0	0.02

- `cos = torch.cat([ang.cos(), ang.cos()], dim=-1)`：先逐元素取 `cos` 得 `[3, 2]`，再和自己拼成 `[3, 4]`（左右两半一模一样）：

位置 p	维0	维1	维2	维3
0	1.0000	1.0000	1.0000	1.0000
1	0.5403	1.0000	0.5403	1.0000
2	-0.4161	0.9998	-0.4161	0.9998

`sin` 同理拼成 `[3, 4]`：

位置 p	维0	维1	维2	维3
0	0.0000	0.0000	0.0000	0.0000
1	0.8415	0.0100	0.8415	0.0100
2	0.9093	0.0200	0.9093	0.0200

（注： $\cos 0.01 \approx 1.0000$ 、 $\cos 0.02 \approx 0.9998$ ，都四舍五入到 4 位小数。）

再跑 `apply_rope(x, cos, sin)`，以位置 1 上的一个向量 `x = [1, 2, 3, 4]` 为例（真实里 `x` 是 `[B, H, L, hd]`，这里只盯住某个 batch、某个 head、位置 1 的那条 4 维向量）：

- `cos, sin = cos[None, None], sin[None, None]`：只是把 `[3, 4]` 变成 `[1, 1, 3, 4]` 好广播，数值不变；位置 1 用的就是上表 `p=1` 那行——`cos = [0.5403, 1.0000, 0.5403, 1.0000]`、`sin = [0.8415, 0.0100, 0.8415, 0.0100]`。

- $x_1, x_2 = x[\dots, :2], x[\dots, 2:]$: 前半 $x_1 = [1, 2]$ 、后半 $x_2 = [3, 4]$ 。配对方式是「前半第 j 维配后半对应的第 $j + \text{hd}/2$ 维」（本例 $\text{hd}=4$ 、 $\text{hd}/2=2$ ，所以是第 0 维配第 2 维、第 1 维配第 3 维）：(1, 3) 一对、(2, 4) 一对。
- `rotated = torch.cat([-x2, x1], dim=-1)` = `[-3, -4, 1, 2]`。
- `return x * cos + rotated * sin`，逐元素算：

维度	<code>x * cos</code>	<code>rotated * sin</code>	相加
0	$1 \times 0.5403 = 0.5403$	$(-3) \times 0.8415 = -2.5245$	-1.9842
1	$2 \times 1.0000 = 2.0000$	$(-4) \times 0.0100 = -0.0400$	1.9600
2	$3 \times 0.5403 = 1.6209$	$1 \times 0.8415 = 0.8415$	2.4624
3	$4 \times 1.0000 = 4.0000$	$2 \times 0.0100 = 0.0200$	4.0200

所以 $x = [1, 2, 3, 4]$ 在位置 1 旋转后变成 `[-1.9842, 1.9600, 2.4624, 4.0200]`。（表里为了好手算，`cos` / `sin` 都四舍五入到了 4 位，逐元素连乘会累积一点舍入误差；真按全精度跑 `apply_rope`，打印出来是 `[-1.9841, 1.9599, 2.4624, 4.0198]`，差在最后一位。）

这串数到底「旋转」了什么？把第 0、2 维挑出来：(1, 3) 变成 (-1.9842, 2.4624)，正好是把二维向量 (1,3) 按上面 `apply_rope` 那个旋转矩阵转了 $\theta_0 \cdot p = 1.0 \times 1 = 1$ 弧度（代入验证： $1 \cdot \cos 1 - 3 \cdot \sin 1 = -1.9842$ 、 $3 \cdot \cos 1 + 1 \cdot \sin 1 = 2.4624$ ✓）。第 1、3 维 (2, 4) 则只转了 $\theta_1 \cdot p = 0.01 \times 1 = 0.01$ 弧度——角度极小，所以几乎没动。这正是 RoPE 的核心：同一个位置 p 下，不同维度对按各自的频率 θ_i 转不同角度；位置 p 越靠后转得越多。拿位置 0 那行验证更直观——`cos` 全 1、`sin` 全 0，`apply_rope` 把 x 原样返回，因为位置 0 不旋转。

四、CausalGQA：因果分组查询注意力

```
class CausalGQA(nn.Module):
    """因果分组查询注意力（第 6、7 章）：H 个 Q 头、G 个 KV 头，含 RoPE 与上三角掩码。"""
    def __init__(self, d_model, n_heads, n_kv_heads):
        super().__init__()
        assert d_model % n_heads == 0 and n_heads % n_kv_heads == 0
        self.n_heads, self.n_kv_heads = n_heads, n_kv_heads
        self.head_dim = d_model // n_heads
        self.q_proj = nn.Linear(d_model, n_heads * self.head_dim, bias=False) # 现代 LLM 投影不带 bias
        self.k_proj = nn.Linear(d_model, n_kv_heads * self.head_dim, bias=False) # GQA: KV 头更少 -> 更窄
        self.v_proj = nn.Linear(d_model, n_kv_heads * self.head_dim, bias=False)
        self.o_proj = nn.Linear(n_heads * self.head_dim, d_model, bias=False)
    def forward(self, x, cos, sin):
        B, L, _ = x.shape
        H, G, hd = self.n_heads, self.n_kv_heads, self.head_dim
        q = self.q_proj(x).view(B, L, H, hd).transpose(1, 2) # [B, H, L, hd]
        k = self.k_proj(x).view(B, L, G, hd).transpose(1, 2) # [B, G, L, hd]
        v = self.v_proj(x).view(B, L, G, hd).transpose(1, 2) # [B, G, L, hd]
        q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)
```



```

rep = H // G
k = k.repeat_interleave(rep, dim=1)          # 复制凑齐 H 份 -> [B, H, L, hd]
v = v.repeat_interleave(rep, dim=1)
scores = (q @ k.transpose(-2, -1)) / math.sqrt(hd)    # [B, H, L, L]
mask = torch.triu(torch.full((L, L), float("-inf")), diagonal=1) # 上三角 -inf
attn = (scores + mask).softmax(dim=-1)
out = attn @ v                                     # [B, H, L, hd]
out = out.transpose(1, 2).contiguous().view(B, L, H * hd) # 合头 -> [B, L, d_model]
return self.o_proj(out)

```

这是本 cell 最重的零件——把第 6、7 章讲的注意力机制连同 GQA、RoPE、因果掩码全实现在一起。

- 注意力（attention）：每个位置用自己的「查询」 Q 去和所有位置的「键」 K 算相似度，得到一组权重，再用这组权重把所有位置的「值」 V 加权求和。一句话：让每个 token 按相关性从别的 token 那里取信息。
- 多头（multi-head）：把上面这套并行做 H 份（ H 个「头」），每个头在一个低维子空间里关注不同的东西，最后拼起来。
- GQA（Grouped-Query Attention，分组查询注意力，第 7 章）： Q 用 H 个头，但 K/V 只用更少的 G 个头（ H 是 G 的整数倍），多个 Q 头共享同一组 K/V 。好处是推理时 KV cache 显著变小、更省显存，几乎不损失效果。
- 因果掩码（causal mask）：语言模型生成时第 t 个 token 只能看见它前面的 token，不能偷看后面。实现办法是在注意力分数矩阵的「上三角」（未来位置）填 $-\infty$ ，softmax 后这些位置权重为 0。

`__init__`：

入参	含义
<code>d_model</code>	模型主干维度
<code>n_heads</code>	Q 头数 H
<code>n_kv_heads</code>	K/V 头数 G （GQA: $G < H$ ）

- `assert d_model % n_heads == 0 and n_heads % n_kv_heads == 0`：两个整除断言——`d_model` 要被 H 整除（好均分成每头 `head_dim`）， H 要被 G 整除（好让每 `H/G` 个 Q 头共享一组 KV）。不满足就直接报错，避免后面形状对不齐。
- `self.head_dim = d_model // n_heads`：每个头的维度 $hd = d_model / H$ 。
- 四个投影层（都 `bias=False`，现代 LLM 的注意力投影一律不带偏置）：查询投影 W_Q （代码里叫 `q_proj`）把 `d_model` 投到 $H*hd$ ；键/值投影 W_K/W_V （`k_proj/v_proj`）只投到 $G*hd$ （更窄，这就是 GQA 省参数 / 省 KV cache 的地方）；输出投影 W_O （`o_proj`）把合头后的 $H*hd$ 投回 `d_model`。

forward: 入参 `x` (`[B, L, d_model]`) 和 RoPE 表 `cos / sin` (`[L, hd]`)，出参 `[B, L, d_model]` (注意力子层形状守恒)。逐步看：

- `B, L, _ = x.shape`：拆出 batch、序列长，第三维 (`d_model`) 用 `_` 接住不用。`H, G, hd = ...` 取出三个超参备用。
- 三个投影 + reshape + 转置：
 - `self.q_proj(x)`：`[B, L, d_model] → [B, L, H*hd]`；`.view(B, L, H, hd)` 把最后一维拆成「头 × 每头维度」`[B, L, H, hd]`；`.transpose(1, 2)` 交换第 1、2 维成 `[B, H, L, hd]`——把「头」提到序列前面，后面好按头并行做注意力。
 - `k / v` 同理，但头数是 `G`，所以是 `[B, G, L, hd]`。
- `q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)`：给 `Q` 和 `K` 注入位置信息 (`V` 不旋转——位置信息只需进到「算相似度」的 `Q·K` 里)。
- `rep = H // G`：每组 `KV` 要被复制几份。`k = k.repeat_interleave(rep, dim=1)`：沿头维 (`dim=1`) 把 `G` 个 `KV` 头各复制 `rep` 份，凑齐成 `H` 份 `[B, H, L, hd]`，这样下面就能和 `H` 个 `Q` 头逐头对齐相乘。`v` 同理。
- `scores = (q @ k.transpose(-2, -1)) / math.sqrt(hd)`：`@` 是矩阵乘。`k.transpose(-2, -1)` 把 `K` 的最后两维转置成 `[B, H, hd, L]`，于是 `q @ kT = [B, H, L, hd] @ [B, H, hd, L] → [B, H, L, L]`：每个头里、每个「查询位置」对每个「键位置」的相似度分数。除以 $\sqrt{hd}$ 是缩放点积注意力的标准做法 (防止维度大时点积过大、softmax 梯度消失)。
- `mask = torch.triu(torch.full((L, L), float("-inf")), diagonal=1)`：先造一个 `[L, L]` 全 $-\infty$ 的矩阵，`torch.triu(..., diagonal=1)` 只保留主对角线以上 (`diagonal=1` 表示从主对角线再往上一格起) 的部分、其余置 0。于是 `mask[i, j] = -inf` 当 `j > i` (未来位置)，否则 0。
- `attn = (scores + mask).softmax(dim=-1)`：把掩码加到分数上 (未来位置变 $-\infty$)，沿最后一维 (键位置) softmax 归一化成权重； $-\infty$ 经 `exp` 后为 0，未来位置权重正好被屏蔽。
- `out = attn @ v`：`[B, H, L, L] @ [B, H, L, hd] → [B, H, L, hd]`，用权重把各位置的 `V` 加权求和。
- `out = out.transpose(1, 2).contiguous().view(B, L, H * hd)`：`.transpose(1, 2)` 把头维换回去 `[B, L, H, hd]`；`.contiguous()` 让内存连续 (`transpose` 后内存不连续，`view` 之前必须先连续化)；`.view(B, L, H*hd)` 把 `H` 个头合并回一条 `[B, L, d_model]`。
- `return self.o_proj(out)`：过输出投影 W_O 投回 `d_model`。

`q @ k.transpose(-2, -1)` 里的 `@` 是「批量矩阵乘」：前面的 `[B, H]` 当 batch 维原样保留，只对最后两维做矩阵乘。`-2 / -1` 是「倒数第 2 / 第 1 维」。

五、SwiGLU：门控前馈网络

```
class SwiGLU(nn.Module):
    """SwiGLU 前馈网络 (第 8 章)：SiLU(gate) ⊙ up，再降维。三个无 bias 线性层。"""
    def __init__(self, d_model, d_ff):
        super().__init__()
```



```

self.gate_proj = nn.Linear(d_model, d_ff, bias=False)
self.up_proj   = nn.Linear(d_model, d_ff, bias=False)
self.down_proj = nn.Linear(d_ff, d_model, bias=False)
def forward(self, x):
    return self.down_proj(F.silu(self.gate_proj(x)) * self.up_proj(x))

```

- FFN（前馈网络）：Transformer 每层在注意力之后还有一个「逐位置」的全连接网络，先把维度从 `d_model` 升到一个更大的中间维 `d_ff`、过激活、再降回 `d_model`。
- SwiGLU（第 8 章）：现代 LLM 常用的 FFN 变体。它把中间层拆成两路——一路 `gate` 过 SiLU 激活当「门」，另一路 `up` 不过激活当「内容」，两路逐元素相乘后再降维。SiLU（也叫 Swish）是 $\text{SiLU}(z) = z \cdot \sigma(z)$ （ σ 是 sigmoid），一条平滑的类 ReLU 曲线。

`__init__`：入参 `d_model`、`d_ff`（中间维）。三个无 bias 线性层：门投影（`gate_proj`）、升维投影（`up_proj`）都把 `d_model` 升到 `d_ff`；降维投影（`down_proj`）把 `d_ff` 降回 `d_model`。

`forward`：入参出参都是 `[B, L, d_model]`（FFN 子层也形状守恒）。那一行 `self.down_proj(F.silu(self.gate_proj(x)) * self.up_proj(x))` 从里往外读：

- `self.gate_proj(x)`：升维 `[..., d_model] → [..., d_ff]`，再 `F.silu(...)` 过 SiLU 激活当「门」。
- `self.up_proj(x)`：另一路升维 `[..., d_model] → [..., d_ff]`，不过激活。
- `*`：两路逐元素相乘（门控）——门接近 0 的地方把内容压住，门大的地方放行。
- `self.down_proj(...)`：把 `d_ff` 降回 `d_model`。

`F.silu` 是函数式写法（直接对张量算，不带可学习参数）；和 `nn.SiLU()` 那种「先建层再调用」等价，这里用 `F.` 更省事。

六、DecoderBlock：把零件组装成一个 Transformer 层

```

class DecoderBlock(nn.Module):
    """一个 Transformer 层：注意力子层 + FFN 子层，各裹 RMSNorm 与残差。
    pre_ln=True 是现代默认 (Pre-LN)；False 切到原版 Post-LN，仅用于第 7.5 节对照实验。"""
    def __init__(self, d_model, n_heads, n_kv_heads, d_ff, pre_ln=True):
        super().__init__()
        self.pre_ln = pre_ln
        self.attn = CausalGQA(d_model, n_heads, n_kv_heads)
        self.ffn = SwiGLU(d_model, d_ff)
        self.norm1 = RMSNorm(d_model)
        self.norm2 = RMSNorm(d_model)
    def forward(self, x, cos, sin):
        if self.pre_ln:                                     # x + Sublayer(Norm(x))
            x = x + self.attn(self.norm1(x), cos, sin)
            x = x + self.ffn(self.norm2(x))
        else:                                               # Norm(x + Sublayer(x))
            x = self.norm1(x + self.attn(x, cos, sin))
            x = self.norm2(x + self.ffn(x))
        return x

```

一个 Transformer 层 (block) = 注意力子层 + FFN 子层, 每个子层都裹着一层 RMSNorm 和一条残差连接 (residual, 把输入直接加到输出上, 让梯度有条「高速公路」直通深层)。

`__init__`: 入参多了个 `pre_ln=True` 开关。建四个零件: `self.attn` (一个 `CausalGQA`)、`self.ffn` (一个 `SwiGLU`)、两个独立的 `RMSNorm` (`norm1` 给注意力子层、`norm2` 给 FFN 子层)。

`forward`: 入参 `x` (`[B, L, d_model]`)、RoPE 表 `cos` / `sin`, 出参同形状 (整个 block 形状守恒, 这是能堆很多层的前提)。两条分支正是正文第 5 节的核心对比:

- Pre-LN (`pre_ln=True`, 现代默认): `x = x + Sublayer(Norm(x))` ——先归一化再进子层, 子层输出加回原始 `x`。残差路径上没有任何 Norm, 是一条「干净的高速公路」, 所以深层也稳、不挑 warmup。
- Post-LN (`pre_ln=False`, 原版 Transformer): `x = Norm(x + Sublayer(x))` ——子层先吃未归一化的 `x`, 输出加回 `x` 后整体再归一化。Norm 卡在残差主干上, 深层容易数值不稳, 强依赖 learning rate warmup。

注意两条分支里 `self.attn(...)` 都要把 `cos, sin` 透传进去 (注意力内部要用 RoPE), `self.ffn(...)` 则不需要。

一句话总结: Cell 2 = 零件车间。把第 6-8 章的四个核心部件——RMSNorm (按均方根缩放)、RoPE (按位置旋转 Q/K)、CausalGQA (H 个 Q 头共享 G 组 KV + 因果掩码)、SwiGLU (门控前馈) ——各用最短代码复现, 再用一个带 `pre_ln` 开关的 `DecoderBlock` 把「注意力子层 + FFN 子层 + RMSNorm + 残差」组装成一个形状守恒的 Transformer 层。本 cell 只定义、不运行。

Cell 3: 拼成完整的 Decoder-only 模型

Cell 2 造好了一个「层」, 这个 cell 把层当积木叠起来, 外面再包上输入侧的 embedding 和输出侧的 final norm + lm_head, 拼出一个完整的 Decoder-only 语言模型 `MiniDecoderLM` ——这就是正文第 2.2 节那条「`input_ids` → embedding → N 层 block → final norm → lm_head」流水线的代码版。

```
# =====
# Cell 3: 把零件拼成完整的 Decoder-only 语言模型
# =====
class MiniDecoderLM(nn.Module):
    """把零件拼成完整的 Decoder-only 语言模型: embedding -> N*block -> final norm -> lm_head."""
    def __init__(self, vocab, d_model=64, n_layers=4, n_heads=4, n_kv_heads=2,
                  d_ff=None, max_len=64, pre_ln=True):
        super().__init__()
        if d_ff is None:
            d_ff = (int(8 / 3 * d_model) + 7) // 8 * 8  # SwiGLU 对齐参数 (约 8/3*d), 取整到 8 的倍数
        self.embed = nn.Embedding(vocab, d_model)  # [V, d] 查表
        self.blocks = nn.ModuleList([
            DecoderBlock(d_model, n_heads, n_kv_heads, d_ff, pre_ln) for _ in range(n_layers)
        ])
        self.final_norm = RMSNorm(d_model)  # 末层归一化 (不属于任何 block)
        self.lm_head = nn.Linear(d_model, vocab, bias=False)  # [d, V] 投回词表
        cos, sin = build_rope_cache(max_len, d_model // n_heads)
```

```

self.register_buffer("cos", cos) # RoPE 表不是参数, 注册成 buffer
self.register_buffer("sin", sin)
def forward(self, ids):
    B, L = ids.shape
    x = self.embed(ids) # [B, L, d]
    cos, sin = self.cos[:L], self.sin[:L]
    for blk in self.blocks:
        x = blk(x, cos, sin) # 每层形状守恒 [B, L, d]
    x = self.final_norm(x)
    return self.lm_head(x) # [B, L, V]

```

`__init__` 入参:

入参	默认	含义
<code>vocab</code>	(必填)	词表大小 V
<code>d_model</code>	64	主干维度
<code>n_layers</code>	4	叠几层 block (N)
<code>n_heads</code>	4	Q 头数 H
<code>n_kv_heads</code>	2	KV 头数 G (GQA)
<code>d_ff</code>	None	FFN 中间维; <code>None</code> 时按规则自动算
<code>max_len</code>	64	RoPE 预备表支持的最大序列长度
<code>pre_ln</code>	True	透传给每个 block 的 Pre/Post-LN 开关

构造逻辑:

- `if d_ff is None: d_ff = (int(8 / 3 * d_model) + 7) // 8 * 8`: 没指定 `d_ff` 就自动算。SwiGLU 因为有两路升维 (gate + up), 为了让总参数量和传统 FFN 相当, 中间维取约 $\frac{8}{3}d_{\text{model}}$; `(... + 7) // 8 * 8` 是「向上取整到 8 的倍数」的常用技巧 (硬件上对齐到 8 更高效)。以 Cell 4 的 `d_model=64` 为例: $\frac{8}{3} \times 64 \approx 170.7$, 向上对齐到 8 的倍数得 `d_ff = 176`。
- `self.embed = nn.Embedding(vocab, d_model)`: token embedding 表, 一张 `[V, d]` 的查找表, 把每个 token id 映射成 `d` 维向量。
- `self.blocks = nn.ModuleList([DecoderBlock(...) for _ in range(n_layers)])`: 用列表推导式造 `n_layers` 个 `DecoderBlock`, 装进 `nn.ModuleList`。 `nn.ModuleList` 是专门存子模块的列表容器——必须用它 (而不是普通 Python `list`), 否则里面的层不会被登记为模型参数、训练时更新不到。 `for _ in range(n_layers)` 里的 `_` 表示「循环变量我不关心, 只要循环 N 次」。
- `self.final_norm = RMSNorm(d_model)`: 最后一层归一化, 作用在最后一个 block 输出、进 `lm_head` 之前。它不属于任何 block, 是单独的一个。

- `self.lm_head = nn.Linear(d_model, vocab, bias=False)`：语言模型头（代码里叫 `lm_head`），把每个位置的 `d` 维向量投回 `v` 维 logits（词表上每个 token 一个分数）。
- `cos, sin = build_rope_cache(max_len, d_model // n_heads)`：调 Cell 2 的函数，按「最大长度 × 每头维度」一次性打好 RoPE 表。
- `self.register_buffer("cos", cos) / ("sin", sin)`：把 RoPE 表登记成 buffer。buffer 和 parameter 的区别是：buffer 不参与梯度更新（RoPE 表是算出来的常量、没有要学的东西），但会跟着模型一起 `.to(device)` 搬到 GPU、一起存进 checkpoint。注册后用 `self.cos` / `self.sin` 取用。

forward：入参 `ids`（`[B, L]` 的 token id 整数矩阵），出参 `logits`（`[B, L, V]`）：

- `B, L = ids.shape`：拆出 batch 和序列长。
- `x = self.embed(ids)`：查表，`[B, L]` 整数 → `[B, L, d]` 向量。
- `cos, sin = self.cos[:L], self.sin[:L]`：从预备好的 RoPE 表里截取前 `L` 个位置（表按 `max_len` 建的，实际序列可能更短）。
- `for blk in self.blocks: x = blk(x, cos, sin)`：依次过每一层 block，每层都 `[B, L, d] → [B, L, d]` 形状守恒，`x` 一路被覆盖更新。
- `x = self.final_norm(x)`：末层归一化。
- `return self.lm_head(x)`：投到词表维，返回 `[B, L, V]`——每个位置、对每个候选 token 的打分。

一句话总结：Cell 3 = 总装车间。MiniDecoderLM 把 Cell 2 的 `DecoderBlock` 用 `nn.ModuleList` 叠 `N` 层，前面接 token embedding、后面接 final RMSNorm + `lm_head`，RoPE 表用 `register_buffer` 存成「不学习但跟着搬家」的常量。`forward` 就是正文第 2.2 节那条流水线：

`ids → embed → N×block（逐层形状守恒）→ final_norm → lm_head → logits`。

Cell 4：前向一遍——逐层形状守恒与参数明细

光定义完模型还不放心，这个 cell 实例化一个小模型、喂一批假数据前向走一遍，干两件事：① 逐层打印形状，亲眼确认「每个 block 形状守恒」（正文第 2.4 节）；② 逐组件数参数量，把正文第 2.3 节那份参数明细落到具体数字上。

```
# =====
# Cell 4: 前向一遍——逐层形状守恒 + 参数明细
# =====
torch.manual_seed(0)
vocab, B, L = 100, 2, 8
model = MiniDecoderLM(vocab, d_model=64, n_layers=4, n_heads=4, n_kv_heads=2)
ids = torch.randint(0, vocab, (B, L))

# 手动走一遍，确认每个 block 都"形状守恒"
x = model.embed(ids)
print("input_ids      :", tuple(ids.shape), ids.dtype)
print("after embed    :", tuple(x.shape))
cos, sin = model.cos[:L], model.sin[:L]
```

```

for i, blk in enumerate(model.blocks):
    x = blk(x, cos, sin)
    print(f"after block[{i}]   :", tuple(x.shape))
x = model.final_norm(x)
print("after final_norm:", tuple(x.shape))
logits = model.lm_head(x)
print("logits           :", tuple(logits.shape), " <- [B, L, vocab]")

def count(m):
    return sum(p.numel() for p in m.parameters())
print("-" * 40)
print("embed  参数:", count(model.embed))
print("1 block 参数:", count(model.blocks[0]),
      "(attn", count(model.blocks[0].attn), "+ ffn", count(model.blocks[0].ffn), ")")
print("lm_head 参数:", count(model.lm_head))
print("整模型 参数:", count(model))

```

第一部分：准备模型和假输入。

- `torch.manual_seed(0)`：固定随机种子，让参数初始化和下面的随机输入每次都一样、结果可复现。
- `vocab, B, L = 100, 2, 8`：用一个示例尺寸——词表 100、batch 2、序列长 8。
- `model = MiniDecoderLM(vocab, d_model=64, n_layers=4, n_heads=4, n_kv_heads=2)`：建一个 4 层、主干维 64、4 个 Q 头 / 2 个 KV 头的小模型。
- `ids = torch.randint(0, vocab, (B, L))`：随机造一批 token id，形状 `[2, 8]`，每个值在 `[0, 100)`。这只是为了跑通形状，不是真数据。

第二部分：手动逐层前向、打印形状。这里没有直接调 `model(ids)`，而是把 `forward` 里的步骤手动摊开，每过一步打印一次形状，好亲眼看到「守恒」：

- `x = model.embed(ids)`：查表 `[2, 8] → [2, 8, 64]`。
- `print("input_ids ...", tuple(ids.shape), ids.dtype)`：打印输入形状和 dtype（会看到 `torch.int64`，token id 是整数）。`tuple(...)` 把 `torch.Size` 转成普通元组，打印更干净。
- `cos, sin = model.cos[:L], model.sin[:L]`：取前 L 个位置的 RoPE 表（和 `forward` 里一致）。
- `for i, blk in enumerate(model.blocks): x = blk(x, cos, sin); print(...)`：依次过每层并打印。`enumerate` 同时给下标 `i` 和层对象 `blk`。预期现象：从 `after embed` 到 `after block[3]` 到 `after final_norm`，形状一直是 `(2, 8, 64)`——这就是「形状守恒」：每个 block 吃 `[B, L, d]`、吐 `[B, L, d]`，所以才能想叠多少层就叠多少层。
- `x = model.final_norm(x) →` 仍是 `(2, 8, 64)`；`logits = model.lm_head(x) → (2, 8, 100)`，只有这最后一步把维度从 `d` 变到 `vocab`。两端（`input_ids` 的 `(2, 8)` 和 `logits` 的 `(2, 8, 100)`）形状不同，中间一路守恒。

第三部分：数参数。

```

def count(m):
    return sum(p.numel() for p in m.parameters())

```

`count` 是个小工具函数：`m.parameters()` 遍历模块 `m` 的所有可学习参数张量，`p.numel()` 数每个张量的元素个数（number of elements），`sum(...)` 全加起来——就是这个模块的总参数量。

- `count(model.embed)`：token embedding 表的参数 = $V \times d = 100 \times 64$ 。
- `count(model.blocks[0])`：第 0 层 block 的总参数，括号里再拆出 `count(...attn)`（注意力四个投影）和 `count(...ffn)`（SwiGLU 三个投影）。你会看到 `ffn` 比 `attn` 大不少——FFN 的参数在 block 里占主要部分（约占一层的 3/4，和第 8 章的结论一致；GQA 让注意力更窄，FFN 占比还更高些）。
- `count(model.lm_head)`：lm_head 的参数 = $d \times V$ 。
- `count(model)`：整个模型的总参数。

把这个具体模型（`vocab=100`、`d_model=64`、`n_layers=4`、`n_heads=4`、`n_kv_heads=2`，于是每头维度 `head_dim=16`、`d_ff=176`、`H·hd=64`、`G·hd=32`）的各组件实际数出来，正好凑成打印的 `197696`：

组件	计算（形状相乘）	参数量
<code>embed</code> （token embedding 表）	$V \times d = 100 \times 64$	6,400
<code>q_proj</code> （ <code>attn</code> ）	$d \times (H \cdot hd) = 64 \times 64$	4,096
<code>k_proj</code> （ <code>attn</code> ，GQA 更窄）	$d \times (G \cdot hd) = 64 \times 32$	2,048
<code>v_proj</code> （ <code>attn</code> ，GQA 更窄）	$d \times (G \cdot hd) = 64 \times 32$	2,048
<code>o_proj</code> （ <code>attn</code> ）	$(H \cdot hd) \times d = 64 \times 64$	4,096
<code>attn</code> 小计（ <code>q+k+v+o</code> ）	$4096+2048+2048+4096$	12,288
<code>gate_proj</code> + <code>up_proj</code> + <code>down_proj</code> （ <code>ffn</code> ）	$3 \times 64 \times 176$	33,792
<code>norm1</code> + <code>norm2</code> （每 block 两个 RMSNorm）	$64 + 64$	128
1 个 block 合计	$12288 + 33792 + 128$	46,208
4 层 block 合计	46208×4	184,832
<code>final_norm</code> （RMSNorm）	64	64
<code>lm_head</code>	$d \times V = 64 \times 100$	6,400
整模型合计	$6400 + 184832 + 64 + 6400$	197,696

① RoPE 的 `cos` / `sin` 是 `register_buffer` 注册的常量、不是参数，所以不计入这 197696；② GQA 让 `k_proj` / `v_proj` 只有 `q_proj` / `o_proj` 的一半宽（各 2048 vs 4096），这正是它省参数（也省 KV cache）的地方；③ `ffn`（33792）的参数在 block 里占主要部分，约占一层的 3/4（精确是

33792 / 46208 $\approx$ 73% ; 真实 Qwen3-8B 上这个比例约 78%, 见 Cell 6) , 和上文「FFN 的参数在 block 里占主要部分」、第 8 章的结论一致。

`sum(p.numel() for p in m.parameters())` 里 `(... for ... in ...)` 是生成器表达式——边遍历边产出每个张量的元素数, 交给 `sum` 累加, 不用先建中间列表, 省内存。

一句话总结: Cell 4 = 前向自检。建一个 4 层小模型, 手动逐层前向并打印形状, 确认从 embedding 到 final norm 全程 `[B, L, d]` 形状守恒 (这正是 Transformer 能堆几十层的结构前提), 只有 `lm_head` 把末维从 `d` 变成 `vocab`; 再用一个 `count` 工具逐组件数参数, 验证「FFN 的参数在一层里占主要部分」。

Cell 5: Pre-LN vs Post-LN 的训练稳定性对照

这是 Cell 5 的核心实验, 给正文第 5 节那个「Norm 放哪儿」的论断一个实验依据。具体做法: 搭一个 24 层的深模型, 分别用 Pre-LN 和 Post-LN, 在一个示例任务上、不加 warmup、用同一个偏大的学习率训练, 看谁能训练; 再补一条「Post-LN + warmup」, 验证 warmup 能不能让 Post-LN 重新正常训练。

```
# =====
# Cell 5: Pre-LN vs Post-LN — 24 层深模型的训练稳定性对照 (CPU 约一两分钟)
# =====
import matplotlib.pyplot as plt

def train_curve(pre_ln, warmup=0, steps=150, n_layers=24, d_model=128,
                n_heads=8, n_kv_heads=2, L=24, lr=2e-3, vocab=64, seed=0):
    """在一个深(24 层)模型上训一个示例任务(把序列循环右移 1 位), 返回 loss 曲线。
    warmup>0 时, 前 warmup 步把 lr 从 0 线性升到 lr (其余步保持 lr)。"""
    torch.manual_seed(seed)
    m = MiniDecoderLM(vocab, d_model=d_model, n_layers=n_layers,
                      n_heads=n_heads, n_kv_heads=n_kv_heads, max_len=L, pre_ln=pre_ln)
    opt = torch.optim.AdamW(m.parameters(), lr=lr)
    losses = []
    for s in range(steps):
        cur_lr = lr * min(1.0, (s + 1) / warmup) if warmup > 0 else lr # 线性 warmup
        for g in opt.param_groups:
            g["lr"] = cur_lr
        ids = torch.randint(0, vocab, (8, L))
        tgt = torch.roll(ids, shifts=1, dims=1) # 确定性映射, 模型学得会
        loss = F.cross_entropy(m(ids).reshape(-1, vocab), tgt.reshape(-1))
        opt.zero_grad(); loss.backward(); opt.step()
        losses.append(loss.item())
    return losses

# 三条曲线 (同样 24 层、同样 lr)。CPU 上约一两分钟。
pre = train_curve(pre_ln=True, warmup=0) # Pre-LN, 不加 warmup
post = train_curve(pre_ln=False, warmup=0) # Post-LN, 不加 warmup
post_wu = train_curve(pre_ln=False, warmup=30) # Post-LN, 加 30 步 warmup
print(f"末步 loss — Pre-LN(无warmup): {pre[-1]:.2f} "
      f"Post-LN(无warmup): {post[-1]:.2f} Post-LN(+warmup): {post_wu[-1]:.2f}")

plt.figure(figsize=(7.5, 4.2))
plt.plot(pre, label="Pre-LN (no warmup)")
plt.plot(post, label="Post-LN (no warmup)")
plt.plot(post_wu, label="Post-LN (+ warmup 30)")
plt.xlabel("training step"); plt.ylabel("loss")
```

```
plt.title("Pre-LN vs Post-LN: training a 24-layer model (no warmup)")
plt.legend(); plt.grid(True, alpha=0.3); plt.tight_layout(); plt.show()
```

- warmup (学习率预热, 见 P04) : 训练刚开始时把学习率从很小 (甚至 0) 线性升到目标值, 走若干步后再保持/衰减。深层、易不稳的模型一上来就用大 lr 容易发散, warmup 让它先小步慢走、等数值稳下来。
- 示例任务「循环右移一位」: 输入一串随机 token, 目标是把它整体向右滚动一格。这是个确定性映射 (同一个输入永远对应同一个输出), 除了位置 0 之外每个位置都能从因果可见的上下文里学会——所以谁的 loss 降不下去, 就是「难收敛」而非「任务太难」, 干净地把变量隔离到了 Pre/Post-LN 上。(位置 0 是唯一例外: 它的目标是被 roll 卷到开头、原本在序列末尾的那个 token, 可因果掩码下位置 0 只看得见自己、看不到末尾, 所以这一个位置天然学不了; 它给 loss 设了一个理论下限 $\approx \frac{1}{L} \ln(\text{vocab}) = \frac{1}{24} \ln 64 \approx 0.17$ ——这正是正文里 Pre-LN 训到最后停在 ≈ 0.2 、而不是干脆到 0 的原因。)

train_curve 函数——训一个模型、返回它的 loss 曲线。入参里关键的是 **pre_ln** (Pre 还是 Post) 和 **warmup** (预热步数, 0 表示不预热), 其余 **steps=150** / **n_layers=24** / **d_model=128** / **L=24** 等是这个实验的固定配置 (24 层是为了「够深」才能暴露出 Post-LN 的不稳)。逐段看:

- **torch.manual_seed(seed)** : 固定种子, 三次调用都从同一个随机起点出发, 保证公平对比。
- **m = MiniDecoderLM(..., pre_ln=pre_ln)** : 按传入的开关建一个 24 层模型。
- **opt = torch.optim.AdamW(m.parameters(), lr=lr)** : AdamW 优化器, 初始学习率 **lr**。
- 训练循环 **for s in range(steps)** :
 - **cur_lr = lr * min(1.0, (s + 1) / warmup) if warmup > 0 else lr** : 算这一步的学习率。
warmup > 0 时, **(s+1)/warmup** 在前 **warmup** 步内从接近 0 线性升到 1 (**min(1.0, ...)** 封顶, 之后一直是 **lr**) ; **warmup == 0** (用三元表达式的 **else** 分支) 则恒为 **lr**、不预热。
 - **for g in opt.param_groups: g["lr"] = cur_lr** : 把算好的学习率写回优化器 (PyTorch 的学习率存在 **param_groups** 里, 手动调度就这么改)。
 - **ids = torch.randint(0, vocab, (8, L))** : 现造一批随机输入 **[8, 24]** (每步都新造, 相当于无限数据)。
 - **tgt = torch.roll(ids, shifts=1, dims=1)** : 目标 = 输入沿序列维 (**dims=1**) 循环右移 1 位。
torch.roll 把末尾滚到开头。
 - **loss = F.cross_entropy(m(ids).reshape(-1, vocab), tgt.reshape(-1))** : 前向 **m(ids)** 得 **[8, 24, vocab]**, **.reshape(-1, vocab)** 摊平成 **[8*24, vocab]** 当一堆独立分类, **tgt.reshape(-1)** 摊成 **[8*24]**, 算交叉熵。
 - **opt.zero_grad(); loss.backward(); opt.step()** : 反向三件套——清梯度、求导、更新。
 - **losses.append(loss.item())** : 记下这步 loss。
- **return losses** : 返回整条 loss 曲线 (长度 **steps**)。

三次调用 + 打印 + 画图:

- **pre = train_curve(pre_ln=True, warmup=0)** : Pre-LN, 不预热。

- `post = train_curve(pre_ln=False, warmup=0)` : Post-LN, 不预热。
- `post_wu = train_curve(pre_ln=False, warmup=30)` : Post-LN, 加 30 步 warmup。
- `print(...)` : 打印三者末步 loss (`{pre[-1]:.2f}` 取曲线最后一个值、保留 2 位小数) 做个速览。
- 后面一段是 matplotlib 画三条 loss 曲线 (`plt.plot(列表, label=...)` 各画一条, `legend` 显示图例, `grid` 加淡网格) 。

预期现象 (也是正文第 5 节结论的实验依据) : Pre-LN (无 warmup) loss 一路平稳下降; Post-LN (无 warmup) 几乎学不会、loss 卡在高位 (≈ 4.2 附近) 几乎不动; Post-LN (+ warmup) 又能正常降下来。三条线放一起, 就把正文第 5 节的结论印证了: Pre-LN 省心 (不挑 warmup 就稳)、Post-LN 难训 (不加 warmup 几乎学不会)、warmup 是稳住 Post-LN 的必需品——这也正是现代大模型默认 Pre-LN 的实验依据。

一句话总结: Cell 5 = 用一个 24 层深模型做控制变量实验。 `train_curve` 在「循环右移」这个确定性示例任务上训一个模型并返回 loss 曲线, 靠 `pre_ln` / `warmup` 两个开关跑出三种配置。结果用数据印证正文第 5 节: 同样深度、同样大 lr 下, Pre-LN 稳、Post-LN 不加 warmup 难收敛、warmup 能让 Post-LN 重新正常训练。

Cell 6: 按真实 Qwen3-8B 尺寸重算参数明细

最后一个 cell 把视角从「示例模型」拉回「真实大模型」: 只下载 Qwen3-8B 的几 KB config (不下权重), 读出它的真实尺寸, 套用我们在正文第 2.3 节推的那套参数公式, 把 ~8B 这个总量一项一项算出来、和全景图对上。

```
# =====
# Cell 6: 读真实 Qwen3-8B config, 按它的尺寸重算参数明细 (对上全景图)
# =====
# AutoConfig 只下载几 KB 的 config.json, 不下几个 GB 的权重, CPU 秒级完成。
from transformers import AutoConfig

cfg = AutoConfig.from_pretrained("Qwen/Qwen3-8B")
V, d = cfg.vocab_size, cfg.hidden_size
H, G = cfg.num_attention_heads, cfg.num_key_value_heads
hd = getattr(cfg, "head_dim", d // H) # Qwen3 显式给出 head_dim; 它不一定等于 d/H (如 Qwen3-4B 就不等)
d_ff, n_layers = cfg.intermediate_size, cfg.num_hidden_layers
tie = cfg.tie_word_embeddings

emb = V * d # token embedding
q = d * (H * hd) # q_proj
kv = 2 * d * (G * hd) # k_proj + v_proj (GQA: KV 头少, 所以更窄)
o = (H * hd) * d # o_proj
attn = q + kv + o
ffn = 3 * d * d_ff # SwiGLU 三个矩阵
per_layer = attn + ffn
blocks = per_layer * n_layers
head = 0 if tie else V * d # 8B 不绑定权重, lm_head 单独一份
total = emb + blocks + head

M = lambda x: f"{x / 1e6:6.0f} M"
print(f"vocab={V} d_model={d} layers={n_layers} H={H} G={G} head_dim={hd} d_ff={d_ff} tie={tie}")
print("-" * 52)
```

```

print("token embedding  :", M(emb))
print("  attn  / layer  :", M(attn), f"(q{M(q)} + kv{M(kv)} + o{M(o)})")
print("  ffn   / layer  :", M(ffn), " <- 约占一层的 3/4")
print(f"  block ×{n_layers}      : {blocks / 1e9:.2f} B")
print("lm_head              :", M(head), "(tied)" if tie else "(untied)")
print("-" * 52)
print(f"TOTAL                  : {total / 1e9:.2f} B  <- 这就是 Qwen3-8B 的 ~8B")

```

- **AutoConfig** (来自 **transformers** 库)：只读取一个模型的配置文件 **config.json** (几 KB)，里面记着层数、维度、头数等超参，但不含权重。所以这一步 CPU 秒级完成，不需要下载几 GB 的模型、也不需要 GPU。

第一部分：读 config，取出尺寸。

- `cfg = AutoConfig.from_pretrained("Qwen/Qwen3-8B")`：联网拉取 Qwen3-8B 的 config (Qwen3 系列要求 `transformers>=4.51`)。
- 接下来几行把关键超参取进局部变量：**V** (词表)、**d** (主干维 `hidden_size`)、**H** (Q 头 `num_attention_heads`)、**G** (KV 头 `num_key_value_heads`)。
- `hd = getattr(cfg, "head_dim", d // H)`：取每头维度。`getattr(对象, "属性名", 默认值)` 是「安全取属性」——config 里有 `head_dim` 就用它，没有就返回 `d // H`。为什么不直接用 `d // H`？因为 Qwen3 显式给出 `head_dim`，而它不一定等于 `d / H` (比如 Qwen3-4B 就不相等)，所以必须以 config 里的为准。
- `d_ff` (FFN 中间维 `intermediate_size`)、`n_layers` (层数 `num_hidden_layers`)、`tie` (`tie_word_embeddings`，是否让 embedding 和 lm_head 共享同一份权重)。

第二部分：套公式算参数。这正是正文第 2.3 节那份参数明细，逐项对应：

- `emb = V * d`：token embedding 表。
- `q = d * (H * hd)`：查询投影 W_Q 的参数 (`d_model × (H·hd)`)。
- `kv = 2 * d * (G * hd)`：键 + 值两个投影，头数是 G (GQA 让它们更窄，这是省参数的关键)。
- `o = (H * hd) * d`：输出投影 W_O 的参数。
- `attn = q + kv + o`：一层注意力总参数。
- `ffn = 3 * d * d_ff`：SwiGLU 的三个矩阵 (gate / up / down)，各 `d × d_ff`。
- `per_layer = attn + ffn`：一层总参数；`blocks = per_layer * n_layers`：所有层。
- `head = 0 if tie else V * d`：lm_head 参数。绑定权重 (`tie=True`) 时 lm_head 复用 embedding、不额外占参数，记 0；否则单独一份 `V × d`。Qwen3-8B 不绑定，所以这里是 `V × d`。
- `total = emb + blocks + head`：全模型总参数。

第三部分：格式化打印。

- `M = lambda x: f"{x / 1e6:6.0f} M"`：一个匿名函数 (lambda)，把参数个数换算成「百万 (M)」单位、占 6 字符宽、不留小数，再加个 **M** 后缀。`lambda 参数: 表达式` 是定义小函数的简写。

- 后面几行 `print` 把各项按「embedding / 每层 attn / 每层 ffn / 所有 block / lm_head / TOTAL」打印出来。其中 `f"...{blocks / 1e9:.2f} B..."` 用十亿 (B) 单位、2 位小数；`"(tied)" if tie else "(untied)"` 又是三元表达式，按是否绑定权重打不同标注。最后那行 TOTAL 会打出约 8.19 B——这就是「Qwen3-8B」名字里 8B 的由来，和正文第 2.3 节全景图上的数字对上了。

一句话总结：Cell 6 = 把参数明细从示例尺寸搬到真实尺寸。用 `AutoConfig` 只读 Qwen3-8B 的几 KB config（不下权重、CPU 秒级），取出真实的层数 / 维度 / 头数，套正文第 2.3 节那套公式逐项算参数，最后 $TOTAL \approx 8.19 \text{ B}$ ——既验证了我们对 Transformer 参数构成的理解，也把抽象的全景图落到了一个真实在跑的大模型上。其中两个易忽略的细节：`head_dim` 要以 config 为准（不一定等于 `d/H`）、lm_head 是否占参数取决于 `tie_word_embeddings`。

至此第九章实战的 Cell 2-6 全部讲完。回顾整条主线：造四个零件并拼出一个 Transformer 层 (Cell 2) → 叠成完整 Decoder-only 模型 (Cell 3) → 前向自检形状守恒与参数明细 (Cell 4) → 24 层深模型印证 Pre-LN / Post-LN 的训练稳定性差异 (Cell 5) → 读真实 Qwen3-8B config 把参数明细对上全景图 (Cell 6)。配合正文第九章：Transformer 整体架构的原理推导一起看，「为什么这样设计」和「代码每一行在干什么」就都齐了。